

Documentación Técnica de la Página Web de Gestión de Inventario del Taller

Introducción

Este documento tiene como objetivo explicar detalladamente el funcionamiento interno y la estructura de la página web desarrollada para la gestión del inventario del taller del instituto. La aplicación web ha sido creada utilizando tecnologías web estándar como HTML5, CSS3, JavaScript y XML, permitiendo representar visualmente la distribución del taller y gestionar los materiales almacenados en él.

La página web permite al usuario:

- Visualizar un mapa interactivo del taller.
- Consultar la ubicación de materiales y componentes.
- Navegar entre armarios, baldas y cajones.
- Mostrar información detallada del inventario.
- Generar informes del inventario en formato HTML.
- Exportar informes en formato CSV a partir de un archivo XML.
- Iniciar sesión con un log in.
- Poder contactar en el apartado de contactos

La estructura del proyecto se encuentra organizada de forma modular, separando cada parte de la aplicación según su función:

- HTML para la estructura de las páginas.
- CSS para el diseño visual y estilos.
- JavaScript para la lógica e interactividad.
- XML para el almacenamiento de los datos del inventario.

Esta organización facilita el mantenimiento del proyecto, mejora la legibilidad del código y permite futuras ampliaciones como sistemas de autenticación, bases de datos o paneles de administración.

1. Documentación Técnica de Inicio.html

El archivo Inicio.html corresponde a la página principal de la aplicación web. Su función es servir como punto de entrada al sistema y proporcionar acceso a las funcionalidades principales relacionadas con los informes del inventario.

1.1 Estructura básica del documento HTML

```
1 <!DOCTYPE html>
2 <html lang="es">
3
4 <head>
5   <meta charset="UTF-8">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <title>Inicio</title>
8   <link rel="icon" type="image/png" href="Martillo.png">
9   <link rel="stylesheet" href="Estilo.css">
10 </head>
```

Explicación técnica

El documento comienza definiendo el tipo de documento HTML5 mediante:

```
<!DOCTYPE html>
```

Esto indica al navegador que la página utiliza el estándar moderno de HTML5.

A continuación, se abre la etiqueta principal:

```
<html lang="es">
```

El atributo lang="es" especifica que el idioma principal de la página es español, lo que mejora la accesibilidad y el posicionamiento web.

Dentro del bloque <head> se definen los metadatos y recursos necesarios para la página:

```
<meta charset="UTF-8">
```

Define la codificación UTF-8, permitiendo mostrar correctamente caracteres especiales y tildes.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Permite que la página se adapte correctamente a distintos tamaños de pantalla.

El título de la página se define mediante:

```
<title>Inicio</title>
```

Este texto aparecerá en la pestaña del navegador.

Posteriormente se carga el icono de la página:

```
<link rel="icon" type="image/png" href="Martillo.png">
```

El favicon mejora la identificación visual de la web en el navegador.

Finalmente se enlaza la hoja de estilos principal:

```
<link rel="stylesheet" href="Estilo.css">
```

Esto permite aplicar todos los estilos definidos en el archivo CSS externo.

1.2 Cabecera de la página (header)

```
12 <body>
13   <header>
14     <figure id="logo">
15       
16     </figure>
17     <h1>Distribución del taller</h1>
18   </header>
19
```

Explicación técnica

La cabecera de la página contiene el logotipo y el título principal del sistema.

La etiqueta header representa la cabecera semántica de la página.

`<header>`

La etiqueta `figure` se utiliza para contener elementos gráficos relacionados, en este caso el logotipo.

`<figure id="logo">`

La imagen se carga mediante la etiqueta `img`.

``

El título principal se muestra utilizando una etiqueta `h1`.

`<h1>Distribución del taller</h1>`

1.3 Menú de navegación (nav)

```
20 <nav>
21   <ul>
22     <li><a href="Inicio.html">Inicio</a></li>
23     <li><a href="Mapa.html">Mapa</a></li>
24     <li><a href="LogIn.html">Log in</a></li>
25     <li><a href="Contacto.html">Contacto</a></li>
26   </ul>
27 </nav>
```

Explicación técnica

La barra de navegación permite desplazarse entre las diferentes páginas del sistema.

`<nav>`

La etiqueta `ul` crea una lista no ordenada para organizar los enlaces del menú.

`<ul>`

Cada opción del menú se encuentra dentro de una etiqueta `li`.

`<li>`

La etiqueta `a` define un enlace hacia otra página.

`<a href="Inicio.html">Inicio</a>`

El menú incluye accesos a Inicio, Mapa, Login y Contacto.

1.4 Contenido principal (main)

```
28 <main>
29   <h2>Información</h2>
30   <p>En esta página se recogen los materiales del aula del tayer de mecánica del instituto IES Miguel Herrero Pereda</p>
31   <div class="botones-informes">
32     <button onclick="generarCSVDesdeXML()">Generar informe CSV</button>
33     <button onclick="mostrarInventarioDesdeXML()">Mostrar inventario HTML</button>
34     <button onclick="ocultarInventario()">Ocultar inventario</button>
35   </div>
36   <div id="inventarioHTML"></div>
37
38 </main>
```

Explicación técnica

La etiqueta main contiene el contenido principal de la página.

```
<main>
```

El encabezado h2 organiza visualmente la sección de información.

```
<h2>Información</h2>
```

El párrafo descriptivo explica la finalidad de la página y del inventario.

```
<p>En esta página se recogen los materiales...</p>
```

1.5 Botones de generación de informes

```
<div class="botones-informes">
  <button onclick="generarCSVDesdeXML()">Generar informe CSV</button>
  <button onclick="mostrarInventarioDesdeXML()">Mostrar inventario HTML</button>
  <button onclick="ocultarInventario()">Ocultar inventario</button>
</div>
```

Explicación técnica

Esta sección contiene tres botones interactivos que ejecutan funciones JavaScript.

```
<div class="botones-informes">
```

El botón Generar informe CSV ejecuta la función generarCSVDesdeXML().

```
<button onclick="generarCSVDesdeXML()">
```

El botón Mostrar inventario HTML ejecuta la función mostrarInventarioDesdeXML().

```
<button onclick="mostrarInventarioDesdeXML()">
```

El botón Ocultar inventario ejecuta la función ocultarInventario().

```
<button onclick="ocultarInventario()">
```

El atributo onclick permite asociar eventos JavaScript a elementos HTML.

1.6 Contenedor dinámico del inventario

```
<div id="inventarioHTML"></div>
```

Explicación técnica

Este contenedor inicialmente se encuentra vacío.

```
<div id="inventarioHTML"></div>
```

Su función es actuar como zona dinámica donde JavaScript insertará la tabla HTML generada desde el XML.

El identificador inventarioHTML permite acceder al elemento desde JavaScript mediante document.getElementById().

```
document.getElementById("inventarioHTML")
```

1.7 Inclusión del archivo JavaScript

```
<script src="informes.js"></script>
```

Explicación técnica

Esta línea carga el archivo JavaScript informes.js.

```
<script src="informes.js"></script>
```

El script se coloca al final del documento para garantizar que todos los elementos HTML hayan sido cargados antes de ejecutar JavaScript.

1.8 Pie de página

```
44 <footer>
45   <a href="#top">
46     <figure>
47       
48     </figure>
49   </a>
50   <p>Grupo 3 &copy;</p>
51 </footer>
```

Explicación técnica

El pie de página de la web se encuentra definido mediante la etiqueta:

```
<footer>
```

La etiqueta footer representa la parte final del documento HTML y normalmente se utiliza para mostrar información adicional, enlaces rápidos o elementos de navegación secundarios.

Dentro del footer se incluye un enlace:

```
<a href="#top">
```

El atributo href="#top" permite desplazar automáticamente la página hasta la parte superior cuando el usuario pulsa sobre el elemento. Esto funciona como un botón de “volver arriba”.

Dentro del enlace se utiliza una etiqueta:

```
<figure>
```

que contiene una imagen:

```

```

La imagen flecha.svg actúa visualmente como botón de retorno al inicio de la página. El atributo width="3%" ajusta el tamaño de la imagen de forma proporcional.

Finalmente, se muestra un texto mediante:

```
<p>Grupo 3 &copy;</p>
```

La etiqueta `p` crea un párrafo de texto.

El símbolo:

`©`

representa el carácter de copyright (©), utilizado para indicar la autoría o propiedad del proyecto.

El footer permite mantener una estructura más organizada y proporciona una zona final común en todas las páginas de la aplicación web.

2. Documentación Técnica de Login.html

El archivo Login.html corresponde a la página de inicio de sesión de la aplicación web. Su objetivo es mostrar un formulario sencillo donde el usuario puede introducir su nombre y contraseña, reiniciar los campos o acceder a la opción de creación de cuenta. Esta página utiliza la hoja de estilos general Estilo.css y carga scripts externos para añadir funcionalidad al formulario.

2.1 Estructura básica del documento HTML

```
1 <!DOCTYPE html>
2 <html lang="es">
3
4 <head>
5   <title>Login</title>
6   <meta charset="UTF-8">
7   <link rel="icon" type="image/png" href="Martillo.png">
8   <link rel="stylesheet" href="Estilo.css">
9   <script src="script.js" defer></script>
10  <script src="login.js" defer></script>
11  <meta name="viewport" content="width=device-width, initial-scale=1.0">
12
```

Explicación técnica

El documento comienza declarando el uso de HTML5 mediante `<!DOCTYPE html>`.

`<!DOCTYPE html>`

La etiqueta `html` incluye `lang="es"`, indicando que el contenido principal está en español.

`<html lang="es">`

Dentro del `head` se define el título de la pestaña del navegador.

`<title>Login</title>`

La etiqueta `meta charset="UTF-8"` permite mostrar correctamente caracteres especiales y tildes.

`<meta charset="UTF-8">`

Se enlaza el icono de la página mediante `Martillo.png`.

```
<link rel="icon" type="image/png" href="Martillo.png">
```

Se enlaza la hoja de estilos Estilo.css para mantener el mismo diseño visual que el resto de páginas.

```
<link rel="stylesheet" href="Estilo.css">
```

También se cargan los archivos JavaScript externos usando defer, para que se ejecuten después de cargar el HTML.

```
<script src="login.js" defer></script>
```

2.2 Cabecera de la página (header)

```
15 <header>
16   <figure id="logo">
17     
18   </figure>
19   <h1>Iniciar sesión</h1>
20 </header>
```

Explicación técnica

La cabecera mantiene la misma estructura visual que las demás páginas del proyecto.

```
<header>
```

El elemento figure con id logo contiene la imagen del logotipo.

```
<figure id="logo">
```

La imagen logo.png representa visualmente la página o el centro.

```

```

El título h1 indica al usuario que se encuentra en la página de inicio de sesión.

```
<h1>Iniciar sesión</h1>
```

2.3 Barra de navegación (nav)

```
22 <nav>
23   <ul>
24     <li><a href="index.html">Inicio</a></li>
25     <li><a href="Mapa.html">Mapa</a></li>
26     <li><a href="LogIn.html">Log in</a></li>
27     <li><a href="Contacto.html">Contacto</a></li>
28   </ul>
29 </nav>
```

Explicación técnica

La barra de navegación permite acceder al resto de secciones del sitio web.

```
<nav>
```

La lista ul organiza los enlaces de navegación.

```
<ul>
```

Cada enlace a utiliza href para indicar la página de destino.

```
<a href="Mapa.html">Mapa</a>
```

El menú contiene enlaces hacia Inicio, Mapa, Log in y Contacto.

2.4 Contenido principal (main)

```
31 <main>
32
33 <h2>Iniciar Sesión</h2>
34 <p>
35
36 <article id="inicioSesion">
37 <section class="seccionForm">
38 <figure>
39 | 
40 </figure>
41 Nombre:
42 <input type="text" class="campo" id="usuario" maxlength="25" placeholder="JuanMasache25">
43 Contraseña:
44 <input type="password" class="campo" id="password" maxlength="25">
45 </section>
46
47 <div class="botones-informes">
48 <button type="submit" id="enviar">Iniciar sesión</button>
49 <button type="reset" id="reinicio">Reiniciar campos</button>
50 <button type="button" id="crearCuenta">Crear cuenta</button>
51 </div>
52 </article>
53
54 <div id="avisos"></div>
55
56 </p>
57
58 </main>
```

Explicación técnica

La etiqueta main contiene el contenido principal de la página de login.

```
<main>
```

El encabezado h2 muestra el título de la sección.

```
<h2>Iniciar Sesión</h2>
```

Dentro del main se encuentra el artículo con el formulario de inicio de sesión.

```
<article id="inicioSesion">
```

2.5 Formulario visual de inicio de sesión


```

36     <article id="inicioSesion">
37         <section class="seccionForm">
38             <figure>
39                 
40             </figure>
41             Nombre:
42             <input type="text" class="campo" id="usuario" maxlength="25" placeholder="JuanMasacre25">
43             Contraseña:
44             <input type="password" class="campo" id="password" maxlength="25">
45         </section>
46
47         <div class="botones-informes">
48             <button type="submit" id="enviar">Iniciar sesión</button>
49             <button type="reset" id="reinicio">Reiniciar campos</button>
50             <button type="button" id="crearCuenta">Crear cuenta</button>
51         </div>
52     </article>

```

Explicación técnica

El article agrupa todo el contenido relacionado con el inicio de sesión.

```
<article id="inicioSesion">
```

La sección seccionForm contiene la imagen de usuario y los campos del formulario.

```
<section class="seccionForm">
```

La imagen usuario.png se utiliza como icono visual del formulario.

```

```

El primer input permite introducir el nombre de usuario.

```
<input type="text" class="campo" id="usuario" maxlength="25"
placeholder="JuanMasacre25">
```

El segundo input permite introducir la contraseña. Al usar type="password", el texto queda oculto.

```
<input type="password" class="campo" id="password" maxlength="25">
```

El atributo maxlength="25" limita la longitud máxima de los datos introducidos.

2.6 Botones del formulario

```

47     <div class="botones-informes">
48         <button type="submit" id="enviar">Iniciar sesión</button>
49         <button type="reset" id="reinicio">Reiniciar campos</button>
50         <button type="button" id="crearCuenta">Crear cuenta</button>
51     </div>

```

Explicación técnica

Los botones se agrupan dentro de la clase botones-informes para reutilizar el estilo ya definido en CSS.

```
<div class="botones-informes">
```

El botón Iniciar sesión tiene id="enviar", que permite identificarlo desde JavaScript.

```
<button type="submit" id="enviar">Iniciar sesión</button>
```

El botón Reiniciar campos tiene id="reinicio" y sirve para limpiar los datos del formulario.

```
<button type="reset" id="reinicio">Reiniciar campos</button>
```

El botón Crear cuenta tiene id="crearCuenta" y puede utilizarse para mostrar o activar una opción de registro.

```
<button type="button" id="crearCuenta">Crear cuenta</button>
```

2.7 Zona de avisos

```
54 <div id="avisos"></div>
```

Explicación técnica

El div avisos está vacío inicialmente.

```
<div id="avisos"></div>
```

Su función es servir como zona dinámica para mostrar mensajes generados mediante JavaScript.

Por ejemplo, puede mostrar errores de usuario vacío, contraseña incorrecta o confirmaciones.

2.8 Pie de página (footer)

```
60 <footer>
61   <a href="#top">
62     <figure>
63       
64     </figure>
65   </a>
66   <p>Grupo 3 &copy;</p>
67 </footer>
```

Explicación técnica

El footer representa la parte final de la página.

```
<footer>
```

Incluye un enlace href="#top" para volver a la parte superior de la página.

```
<a href="#top">
```

La imagen flecha.png funciona como botón visual para subir al inicio.

```

```

El texto Grupo 3 © indica la autoría del proyecto.

<p>Grupo 3 ©</p>

3. Documentación Técnica de Mapa.html

El archivo Mapa.html corresponde a la página encargada de representar visualmente la distribución del taller y permitir la localización interactiva de materiales dentro del inventario. Esta página trabaja conjuntamente con el archivo javas.js, encargado de generar dinámicamente el mapa y gestionar toda la interactividad del sistema.

3.1 Estructura básica del documento HTML

```
1  <!DOCTYPE html>
2  <html lang="es">
3
4  <head>
5      <meta charset="UTF-8">
6      <meta name="viewport" content="width=device-width, initial-scale=1.0">
7      <title>Inventario Taller</title>
8
9      <link rel="icon" type="image/png" href="Martillo.png">
10     <link rel="stylesheet" href="Estilo.css">
11 </head>
```

Explicación técnica

El documento comienza definiendo el estándar HTML5 mediante la etiqueta <!DOCTYPE html>.

<!DOCTYPE html>

La etiqueta html define el idioma principal de la página como español.

<html lang="es">

Dentro del bloque head se cargan los metadatos y recursos necesarios.

<head>

La hoja de estilos Estilo.css se enlaza para aplicar el diseño visual de la página.

<link rel="stylesheet" href="Estilo.css">

También se carga el icono de la página mediante el favicon.

<link rel="icon" type="image/png" href="Martillo.png">

3.2 Cabecera de la página (header)

```

15     <header>
16         <figure id="logo">
17             
18         </figure>
19         <h1>Mapa</h1>
20     </header>

```

Explicación técnica

La cabecera contiene el logo y el título principal de la página del mapa.

`<header>`

La etiqueta figure se utiliza para agrupar la imagen del logotipo.

`<figure id="logo">`

La imagen logo.png representa visualmente el sistema.

``

El encabezado h1 muestra el título principal "Mapa".

`<h1>Mapa</h1>`

3.3 Barra de navegación (nav)

```

22     <nav>
23         <ul>
24             <li><a href="Inicio.html">Inicio</a></li>
25             <li><a href="Mapa.html">Mapa</a></li>
26             <li><a href="Contacto.html">Contáctanos</a></li>
27             <li><a href="Login.html">Login</a></li>
28         </ul>
29     </nav>

```

Explicación técnica

La barra de navegación permite desplazarse entre las diferentes páginas del proyecto.

`<nav>`

La lista ul organiza los enlaces del menú.

`<ul>`

Cada opción del menú se encuentra dentro de un elemento li.

`<li>`

Los enlaces a utilizan el atributo href para redirigir a otras páginas HTML.

`<a href="Inicio.html">Inicio</a>`

El menú incluye accesos a Inicio, Mapa, Contáctanos y Login.

3.4 Contenedor principal

Explicación técnica

El contenedor principal organiza la página en dos zonas principales.

```
<div class="contenedor">
```

La parte izquierda contiene el menú lateral de componentes.

La parte derecha contiene el mapa interactivo y la zona de información.

La distribución visual se realiza posteriormente mediante Flexbox en CSS.

3.5 Menú lateral de componentes (aside)

```

33 <aside>
34 <h2>Componentes</h2>
35
36 <ul>
37 <li><button onclick="buscarMaterial('Arduino UNO')">Arduino UNO</button></li>
38 <li><button onclick="buscarMaterial('Cable HDMI')">Cable HDMI</button></li>
39 <li><button onclick="buscarMaterial('Raspberry Pi')">Raspberry Pi</button></li>
40 <li><button onclick="buscarMaterial('Protoboard')">Protoboard</button></li>
41 <li><button onclick="buscarMaterial('Resistencias')">Resistencias</button></li>
42
43 <li><button onclick="buscarMaterial('Cable VGA')">Cable VGA</button></li>
44 <li><button onclick="buscarMaterial('Cable SATA')">Cable SATA</button></li>
45 <li><button onclick="buscarMaterial('Cable IDE')">Cable IDE</button></li>
46 <li><button onclick="buscarMaterial('Fuente ATX')">Fuente ATX</button></li>
47 <li><button onclick="buscarMaterial('Disco HDD')">Disco HDD</button></li>
48
49 <li><button onclick="buscarMaterial('Disco SSD')">Disco SSD</button></li>
50 <li><button onclick="buscarMaterial('Memoria RAM DDR4')">Memoria RAM DDR4</button></li>
51 <li><button onclick="buscarMaterial('Procesador Intel')">Procesador Intel</button></li>
52 <li><button onclick="buscarMaterial('Procesador AMD')">Procesador AMD</button></li>
53 <li><button onclick="buscarMaterial('Tarjeta Gráfica')">Tarjeta Gráfica</button></li>
54
55 <li><button onclick="buscarMaterial('Placa Base')">Placa Base</button></li>
56 <li><button onclick="buscarMaterial('Ventilador PWM')">Ventilador PWM</button></li>
57 <li><button onclick="buscarMaterial('Pasta Térmica')">Pasta Térmica</button></li>
58 <li><button onclick="buscarMaterial('Router Cisco')">Router Cisco</button></li>
59 <li><button onclick="buscarMaterial('Switch 24 Puertos')">Switch 24 Puertos</button></li>
60
61 <li><button onclick="buscarMaterial('Crimpadora RJ45')">Crimpadora RJ45</button></li>
62 <li><button onclick="buscarMaterial('Conectores RJ45')">Conectores RJ45</button></li>
63 <li><button onclick="buscarMaterial('Tester de Red')">Tester de Red</button></li>
64 <li><button onclick="buscarMaterial('Multímetro')">Multímetro</button></li>
65 <li><button onclick="buscarMaterial('Soldador')">Soldador</button></li>
66
67 <li><button onclick="buscarMaterial('Estanco')">Estanco</button></li>
68 <li><button onclick="buscarMaterial('Pinzas ESD')">Pinzas ESD</button></li>
69 <li><button onclick="buscarMaterial('Pulsera Antiestática')">Pulsera Antiestática</button></li>
70 <li><button onclick="buscarMaterial('Sensor DHT11')">Sensor DHT11</button></li>
71 <li><button onclick="buscarMaterial('Sensor Ultrasonico')">Sensor Ultrasonico</button></li>
72
73 <li><button onclick="buscarMaterial('LED Rojo')">LED Rojo</button></li>
74 <li><button onclick="buscarMaterial('LED Azul')">LED Azul</button></li>
75 <li><button onclick="buscarMaterial('Pantalla LCD 16x2')">Pantalla LCD 16x2</button></li>
76 <li><button onclick="buscarMaterial('ESP32')">ESP32</button></li>
77 <li><button onclick="buscarMaterial('NodeMCU')">NodeMCU</button></li>
78 </ul>
79 </aside>

```

Explicación técnica

La etiqueta aside representa contenido lateral relacionado con el contenido principal.

```
<aside>
```

Dentro del menú lateral se muestran botones para buscar materiales específicos.

```
<button onclick="buscarMaterial('Arduino UNO')">
```

Cada botón ejecuta la función `buscarMaterial()` definida en `javas.js`.

Cuando el usuario pulsa un componente, el sistema localiza automáticamente su posición dentro del mapa.

3.6 Zona principal del mapa

```
84      <div class="mapa">
85      |
86      |
87      |
88      |   <div id="tituloMapa" class="titulo-mapa">
89      |   |   Vista general del armario
90      |   </div>
91      |
92      |   <div id="mapaDinamico" class="mapa-dinamico">
93      |   </div>
94      |
95      </div>
96
97      <div class="info" id="info">
98      |   Pulsa en un armario, balda, cajón o componente.
99      </div>
100     </main>
101
102   </div>
```

Explicación técnica

La clase `mapa` contiene toda la representación visual del taller.

```
<div class="mapa">
```

El título dinámico del mapa se almacena en el `div` con `id` `tituloMapa`.

```
<div id="tituloMapa" class="titulo-mapa">
```

El contenido interactivo del mapa se genera dinámicamente dentro de `mapaDinamico`.

```
<div id="mapaDinamico" class="mapa-dinamico">
```

El HTML del mapa no está escrito directamente en la página, sino que es generado mediante JavaScript.

3.7 Zona de información

```
97      <div class="info" id="info">
98      |   Pulsa en un armario, balda, cajón o componente.
99      </div>
```

Explicación técnica

La zona de información muestra datos del armario, balda, cajón o material seleccionado.

```
<div class="info" id="info">
```

Inicialmente muestra un mensaje indicando al usuario que puede pulsar elementos del mapa.

El contenido de esta caja se modifica dinámicamente desde JavaScript mediante innerHTML.

```
document.getElementById("info").innerHTML
```

3.8 Pie de página (footer)

```
104 <footer>
105   <a href="#top">
106     <figure>
107       
108     </figure>
109   </a>
110   <p>Grupo 3 &copy;</p>
111 </footer>
```

Explicación técnica

El pie de página contiene un botón visual para volver al inicio de la página.

```
<footer>
```

La imagen flecha.svg actúa como acceso rápido hacia la parte superior.

```

```

El texto Grupo 3 © indica la autoría del proyecto.

```
<p>Grupo 3 &copy;</p>
```

3.9 Inclusión del archivo JavaScript

```
114 <script src="javas.js"></script>
```

Explicación técnica

El archivo javas.js contiene toda la lógica del mapa interactivo.

```
<script src="javas.js"></script>
```

Este script genera dinámicamente armarios, baldas, cajones y materiales.

El script se coloca al final del body para garantizar que todos los elementos HTML hayan sido cargados antes de ejecutar JavaScript.

4. Documentación Técnica de Contacto.html

El archivo Contacto.html corresponde a la página de contacto de la aplicación web. Su función es permitir que el usuario introduzca un correo electrónico, seleccione un asunto y escriba un mensaje. Esta página mantiene la misma estructura visual que el resto del proyecto, utilizando la hoja de estilos Estilo.css y los elementos comunes de cabecera, navegación, contenido principal y pie de página.

4.1 Estructura básica del documento HTML

```
1 <!DOCTYPE html>
2 <html lang="es">
3
4   <head>
5     <title>Login</title>
6     <meta charset="UTF-8">
7     <link rel="icon" type="image/png" href="Martillo.png">
8     <link rel="stylesheet" href="Estilo.css">
9     <script src="script.js" defer></script>
10    <meta name="viewport" content="width=device-width, initial-scale=1.0">
11  
```

Explicación técnica

El documento comienza indicando que se utiliza HTML5 mediante la declaración `<!DOCTYPE html>`.

`<!DOCTYPE html>`

La etiqueta `html` incluye el atributo `lang="es"`, indicando que el idioma principal de la página es español.

`<html lang="es">`

En el bloque `head` se define el título, la codificación UTF-8, el icono de la página y la hoja de estilos externa.

`<head>`

La línea del CSS enlaza Contacto.html con el archivo Estilo.css para mantener el mismo diseño que el resto de páginas.

`<link rel="stylesheet" href="Estilo.css">`

También se carga `script.js` con `defer`, lo que permite cargar el script sin bloquear la carga del HTML.

`<script src="script.js" defer></script>`

4.2 Cabecera de la página

```
14   <header>
15     <figure id="logo">
16       
17     </figure>
18     <h1>Iniciar sesión</h1>
19   </header>
```

Explicación técnica

La cabecera contiene el logotipo y el título visible de la página.

`<header>`

El logotipo se incluye dentro de una etiqueta figure con el identificador logo.

`<figure id="logo">`

La imagen logo.png representa visualmente el proyecto.

``

El título de la cabecera se muestra con una etiqueta h1.

`<h1>Iniciar sesión</h1>`

Aunque la página es de contacto, el título del header aparece como “Iniciar sesión”, por lo que podría corregirse a “Contacto” para que coincida con el contenido de la página.

4.3 Menú de navegación

```
21      <nav>
22      |   <ul>
23      |       <li><a href="index.html">Inicio</a></li>
24      |       <li><a href="Mapa.html">Mapa</a></li>
25      |       <li><a href="LogIn.html">Log in</a></li>
26      |       <li><a href="Contacto.html">Contacto</a></li>
27      |   </ul>
28      </nav>
```

Explicación técnica

El menú de navegación permite moverse entre las distintas páginas del sitio web.

`<nav>`

La lista ul organiza los enlaces principales.

`<ul>`

Cada enlace se crea con la etiqueta a y el atributo href, que indica el archivo HTML al que se dirige.

`<a href="Contacto.html">Contacto</a>`

El menú incluye enlaces a Inicio, Mapa, Log in y Contacto.

4.4 Contenido principal de contacto

```

30 <main>
31
32 <h2>Contacto</h2>
33 <p>
34
35 <article id="inicioSesion">
36
37 <!--Espacio para poner un email-->
38 <section class="seccionForm">
39     Introduce tu email:
40     <input type="email" class="campo">
41
42 <!--Desplegable con motivos de comunicacion-->
43
44     Elige un asunto :
45     <select name="Asunto">
46         <option value="sug">Sugerencia</option>
47         <option value="rep">Reporte de error</option>
48         <option value="com">Comentario</option>
49         <option value="otr">Otro</option>
50     </select>
51
52 <!--Espacio para escribir-->
53
54     Escribe aquí tu mensaje :
55     <textarea rows="10" cols="40" name="texto"></textarea>
56 </section>
57 <!--Boton enviar-->
58 <div class="botones-informes">
59     <button type="submit" id="enviarMensaje">Enviar</button>
60 </div>
61 </article>
62
63 </p>
64
65 </main>

```

Explicación técnica

La etiqueta main contiene el contenido principal de la página de contacto.

<main>

El encabezado h2 identifica la sección como "Contacto".

<h2>Contacto</h2>

Dentro del main se utiliza un article para agrupar el formulario de contacto.

<article id="inicioSesion">

Aunque el identificador se llama inicioSesion, en esta página se reutiliza para contener el formulario de contacto.

4.5 Campo de correo electrónico

40

```
<input type="email" class="campo">
```

Explicación técnica

Este bloque permite al usuario introducir su correo electrónico.

Introduce tu email:

El input de tipo email está pensado para recibir direcciones de correo electrónico.

```
<input type="email" class="campo">
```

La clase campo permite aplicar estilos desde el archivo CSS.

```
class="campo"
```

4.6 Selector de asunto

45

46

47

48

49

50

```
<select name="Asunto">
  <option value="sug">Sugerencia</option>
  <option value="rep">Reporte de error</option>
  <option value="com">Comentario</option>
  <option value="otr">Otro</option>
</select>
```

Explicación técnica

El select crea un menú desplegable para elegir el motivo del mensaje.

```
<select name="Asunto">
```

Cada option representa una opción disponible dentro del desplegable.

```
<option value="sug">Sugerencia</option>
```

Las opciones disponibles son sugerencia, reporte de error, comentario u otro.

4.7 Área de mensaje

55

```
<textarea rows="10" cols="40" name="texto"></textarea>
```

Explicación técnica

El textarea permite al usuario escribir un mensaje más largo.

```
<textarea rows="10" cols="40" name="texto"></textarea>
```

El atributo rows indica la altura del área de texto en líneas.

```
rows="10"
```

El atributo cols indica el ancho aproximado del campo de texto.

```
cols="40"
```

4.8 Botón de envío

```
58 <div class="botones-informes">
59 <button type="submit" id="enviarMensaje">Enviar</button>
60 </div>
```

Explicación técnica

El botón Enviar está pensado para mandar el mensaje escrito por el usuario.

```
<button type="submit" id="enviarMensaje">Enviar</button>
```

El identificador enviarMensaje permite localizar este botón desde JavaScript si se quiere añadir funcionalidad posteriormente.

```
id="enviarMensaje"
```

Actualmente el HTML solo define el botón; para que envíe datos realmente sería necesario conectarlo a JavaScript o a un backend.

4.9 Pie de página

```
67 <footer>
68   <a href="#top">
69     <figure>
70       
71     </figure>
72   </a>
73   <p>Grupo 3 &copy;</p>
74 </footer>
```

Explicación técnica

El footer mantiene la misma estructura que el resto de páginas del proyecto.

```
<footer>
```

Incluye una imagen de flecha que funciona como acceso para volver a la parte superior de la página.

```

```

El texto Grupo 3 © indica la autoría del proyecto.

```
<p>Grupo 3 &copy;</p>
```

5. Documentación Técnica de Estilo.css

El archivo Estilo.css contiene todos los estilos visuales utilizados por la página web. Su función es controlar la apariencia gráfica del sistema, incluyendo colores, tamaños, distribución de elementos, animaciones y efectos visuales. La hoja de estilos permite separar completamente el diseño visual de la estructura HTML, facilitando el mantenimiento y la reutilización del código.

5.1 Reset general de estilos

```
1  * {
2    margin: 0;
3    padding: 0;
4    box-sizing: border-box;
5  }
```

Explicación técnica

El selector universal * aplica estilos a todos los elementos de la página.

* {

margin: 0 elimina los márgenes automáticos del navegador.

margin: 0;

padding: 0 elimina los espacios internos por defecto.

padding: 0;

box-sizing: border-box hace que el tamaño de los elementos incluya padding y bordes.

box-sizing: border-box;

5.2 Variables CSS (:root)

```
7  :root {
8    --azul: #0040a0;
9    --azul-claro: #b2d1ff;
10   --blanco: #ffffff;
11   --negro: #000000;
12   --gris: #252525;
13   --verde: #1bb123;
14   --fuente-principal: Arial, sans-serif;
15 }
```

Explicación técnica

La pseudo clase :root permite definir variables CSS reutilizables.

:root {

Las variables almacenan colores principales utilizados en toda la página.

--azul: #0040a0;

También se definen colores secundarios, fondo y fuente principal.

--fuente-principal: Arial, sans-serif;

El uso de variables facilita modificar el diseño de forma centralizada.

5.3 Estilos generales del body

```

109  body {
110      display: flex;
111      flex-direction: column;
112      min-height: 100vh;
113      font-family: var(--fuente-principal);
114      background-image: url(fondo.jpg);
115      background-size: cover;
116      background-position: center;
117      background-repeat: no-repeat;
118      color: var(--blanco)
119  }

```

Explicación técnica

La etiqueta body define el estilo general de toda la página web.

body {

display: flex organiza los elementos principales utilizando Flexbox.

display: flex;

flex-direction: column coloca los elementos verticalmente.

flex-direction: column;

min-height: 100vh hace que la página ocupe toda la altura de la ventana.

min-height: 100vh;

background-image establece una imagen de fondo para la web.

background-image: url(fondo.jpg);

background-size: cover ajusta la imagen al tamaño completo de la pantalla.

background-size: cover;

font-family aplica la fuente principal definida en las variables CSS.

font-family: var(--fuente-principal);

5.4 Cabecera de la página (header)

```

22  header {
23      display: flex;
24      align-items: center;
25      position: relative;
26      color: var(--blanco);
27      background-color: rgba(0, 0, 0, 0.9);
28      padding: 0px;
29      transform: translateY(-10%);
30
31  }

```

Explicación técnica

El bloque header define el diseño visual de la cabecera.

`header {`

display: flex alinea horizontalmente el logo y el título.

`display: flex;`

align-items: center centra verticalmente los elementos.

`align-items: center;`

background-color establece un fondo oscuro semitransparente.

`background-color: rgba(0, 0, 0, 0.9);`

La propiedad position: relative permite posicionar elementos internos.

`position: relative;`

5.5 Logo y título principal

```
41 #logo img {
42   width: 120px;
43   height: 120px;
44   object-fit: contain;
45 }
46
47 header h1 {
48   font-size: 50px;
49   position: absolute;
50   left: 50%;
51   transform: translateX(-50%) translateY(30%);
52   top: 20px;
53   color: var(--azul-claro)
54 }
```

Explicación técnica

El selector #logo controla la posición del logotipo.

`#logo {`

La imagen del logo utiliza object-fit: contain para mantener proporciones.

`object-fit: contain;`

El título principal se centra horizontalmente utilizando transform.

`transform: translateX(-50%);`

El color del título utiliza la variable azul claro.

`color: var(--azul-claro);`

5.6 Barra de navegación (nav)

```

56  nav {
57      width: 100%;
58      background-color: var(--negro);
59      padding: 10px;
60      margin-bottom: 20px;
61      transform: translateY(-30%)
62  }
63
64  nav ul {
65      display: flex;
66      justify-content: space-evenly;
67      align-items: center;
68      list-style: none;
69      margin: 0;
70      padding: 0;
71  }
72
73  nav li {
74      flex: 1;
75      text-align: center;
76  }
77
78  nav a {
79      display: block;
80      padding: 10px 0;
81      text-decoration: none;
82      color: white;
83      background-color: var(--negro);
84      font-size: 35px;
85      transition: all 0.3s ease;
86  }
87
88  nav a:hover {
89      color: var(--azul-claro);
90      background:
91          linear-gradient(180deg,
92              var(--negro) 0%,
93              var(--negro) 40%,
94              var(--azul) 120%);
95      transform: translateY(-2px);
96      box-shadow: 0 3px 0 var(--azul-claro);
97  }

```

Explicación técnica

La barra de navegación ocupa todo el ancho de la página.

nav {

La lista ul utiliza Flexbox para distribuir los enlaces.

`display: flex;`

`justify-content: space-evenly` reparte los enlaces uniformemente.

`justify-content: space-evenly;`

Los enlaces a eliminan el subrayado mediante `text-decoration: none`.

`text-decoration: none;`

Los enlaces tienen efectos hover con animaciones y degradados.

`nav a:hover {`

5.7 Contenido principal (main)

```
121  main {
122      display: flexbox;
123      margin-left: 20px;
124      margin-right: 20px;
125      flex: 1;
126      background-color: rgba(0, 0, 0, 0.9);
127      padding: 20px;
128  }
129
130  main h2 {
131      text-align: center;
132      margin-bottom: 25px;
133  }
```

Explicación técnica

La etiqueta main contiene el contenido principal de las páginas.

`main {`

El fondo utiliza un color negro semitransparente.

`background-color: rgba(0, 0, 0, 0.9);`

`padding` añade separación interna entre contenido y bordes.

`padding: 20px;`

`flex: 1` permite que el contenido ocupe el espacio disponible.

`flex: 1;`

5.8 Estilos del mapa interactivo

```

135 .mapa {
136     border: 3px solid white;
137     background-color: black;
138     padding: 20px;
139     width: 100%;
140
141     height: 800px;
142     overflow: auto;
143 }
144
145 .mapa-dinamico {
146     width: 100%;
147     height: 500px;
148
149     display: flex;
150     justify-content: center;
151     align-items: flex-start;
152 }

```

Explicación técnica

La clase `.mapa` define el contenedor principal del mapa.

`.mapa {`

`overflow: auto` permite desplazamiento cuando el contenido supera el tamaño visible.

`overflow: auto;`

La clase `.mapa-dinamico` centra visualmente el contenido generado por JavaScript.

`.mapa-dinamico {`

`justify-content: center` centra horizontalmente el mapa.

`justify-content: center;`

5.9 Armarios, baldas y cajones

```

166 .armario-general {
167     width: 90%;
168     max-width: 1100px;
169     border: 3px solid white;
170     padding: 15px;
171 }
172
173 .fila {
174     display: flex;
175     width: 100%;
176 }
177

```

```

178  .zona {
179      border: 2px solid white;
180      background-color: black;
181      margin: 4px;
182      min-height: 55px;
183      font-weight: bold;
184      cursor: pointer;
185
186      display: flex;
187      align-items: center;
188      justify-content: center;
189      transition:
190          background-color 0.35s ease,
191          box-shadow 0.35s ease,
192          border-color 0.35s ease,
193          color 0.35s ease,
194          transform 0.25s ease;
195  }
196
197  .zona:hover {
198      border-color: var(--azul-claro);
199      box-shadow: inset 0 -25px 35px -20px var(--azul);
200      transform: translateY(-2px);
201  }
202
203  .zona-activa {
204      background-color: var(--azul);
205      color: var(--blanco);
206      border-color: var(--azul-claro);
207  }
208
209  .zona-activa:hover {
210      background-color: #002b70;
211      box-shadow: inset 0 -25px 35px -20px var(--azul-claro);
212  }

```

```

214  .balda {
215      flex: 1;
216  }
217
218  .balda-larga {
219      width: 100%;
220  }

```

```

239  .cajon {
240      flex: 1;
241      min-height: 60px;
242      background-color: black;
243  }

```

Explicación técnica

La clase .armario-general organiza la estructura completa del mapa.

```
.armario-general {
```

La clase .fila distribuye elementos horizontalmente mediante Flexbox.

```
.fila {
```

La clase .zona es la base visual para armarios, baldas y cajones.

```
.zona {
```

cursor: pointer indica que los elementos son interactivos.

```
cursor: pointer;
```

transition añade animaciones suaves al interactuar con las zonas.

```
transition: background-color 0.35s ease;
```

La clase .cajon representa visualmente los cajones del inventario.

```
.cajon {
```

5.10 Efectos visuales y selección

```

197  .zona:hover {
198      border-color: var(--azul-claro);
199      box-shadow: inset 0 -25px 35px -20px var(--azul);
200      transform: translateY(-2px);
201  }
202
203  .zona-activa {
204      background-color: var(--azul);
205      color: var(--blanco);
206      border-color: var(--azul-claro);
207  }
208
209  .zona-activa:hover {
210      background-color: #002b70;
211      box-shadow: inset 0 -25px 35px -20px var(--azul-claro);
212  }

```

Explicación técnica

La pseudo clase :hover aplica efectos visuales al pasar el ratón.

```
.zona:hover {
```

box-shadow crea efectos de iluminación internos.

```
box-shadow: inset 0 -25px 35px -20px var(--azul);
```

transform: translateY(-2px) crea un pequeño movimiento visual.

```
transform: translateY(-2px);
```

La clase .zona-activa resalta la zona seleccionada en azul.

```
.zona-activa {
```

5.11 Menú lateral (aside)

```

290   aside {
291     min-width: 250px;
292     margin-left: 20px;
293     width: 15%;
294     background-color: rgba(0, 0, 0, 0.9);
295     padding: 20px;
296     max-height: 90vh;
297     overflow-y: auto;
298   }
299
300   aside h2 {
301     text-align: center;
302     margin-bottom: 25px;
303   }
304
305   aside ul {
306     padding: 0;
307     list-style: none;
308   }
309
310   aside li {
311     margin-bottom: 14px;
312   }
313
314   aside button {
315     width: 100%;
316     padding: 10px 0;
317     cursor: pointer;
318
319     background-color: var(--negro);
320     color: var(--blanco);
321
322     border: 2px solid var(--azul);
323
324     font-weight: bold;
325
326     transition: all 0.2s ease;
327   }
328
329   aside button:hover {
330
331     color: var(--azul-claro);
332
333     background:
334       linear-gradient(180deg,
335         var(--negro) 0%,
336         var(--negro) 75%,
337         var(--azul) 100%);
338
339     transform: translateY(-2px);
340
341     border-color: var(--azul-claro);
342   }
343
344   aside::-webkit-scrollbar {
345     width: 8px;
346   }
347
348   aside::-webkit-scrollbar-track {
349     background: rgba(0, 0, 0, 0.5);
350   }
351
352   aside::-webkit-scrollbar-thumb {
353     background: var(--azul);
354   }
355
356   aside::-webkit-scrollbar-thumb:hover {
357     background: var(--azul-claro);
358   }

```

Explicación técnica

La etiqueta aside contiene el listado lateral de componentes.

`aside {`

`overflow-y: auto` permite desplazamiento vertical.

`overflow-y: auto;`

Los botones laterales utilizan estilos personalizados.

`aside button {`

Los botones tienen efectos hover similares al menú principal.

`aside button:hover {`

5.12 Caja de información

```
360  .info {
361      margin-top: 30px;
362
363      height: 220px;
364      overflow-y: auto;
365
366      border: 2px solid white;
367      padding: 15px;
368  }
369
370  .info h3 {
371      margin-bottom: 18px;
372  }
373  .info::-webkit-scrollbar {
374      width: 8px;
375  }
376
377  .info::-webkit-scrollbar-track {
378      background: rgba(0,0,0,0.5);
379  }
380
381  .info::-webkit-scrollbar-thumb {
382      background: var(--azul);
383  }
384
385  .info::-webkit-scrollbar-thumb:hover {
386      background: var(--azul-claro);
387  }
```

Explicación técnica

La clase `.info` muestra información sobre materiales y ubicaciones.

`.info {`

height define una altura fija para la caja.

`height: 220px;`

overflow-y: auto añade desplazamiento vertical automático.

`overflow-y: auto;`

También se personalizan las barras de desplazamiento.

`.info::-webkit-scrollbar {`

5.13 Botones de informes

```
402  ✓ .botones-informes button {
403      padding: 12px 20px;
404
405      background-color: var(--negro);
406      color: var(--blanco);
407
408      border: 2px solid var(--azul);
409
410      cursor: pointer;
411
412      transition: all 0.25s ease;
413  }
414
415
416  ✓ .botones-informes button:hover {
417      color: var(--azul-claro);
418
419      background:
420      linear-gradient(
421          180deg,
422          var(--negro) 0%,
423          var(--negro) 75%,
424          var(--azul) 100%
425      );
426
427      transform: translateY(-2px);
428
429      border-color: var(--azul-claro);
430  }
431  }
```

Explicación técnica

La clase .botones-informes organiza los botones mediante Flexbox.

`.botones-informes {`

gap añade separación horizontal entre botones.

gap: 20px;

Los botones utilizan colores oscuros y bordes azules.

border: 2px solid var(--azul);

Los botones incluyen efectos hover con degradados y animaciones.

.botones-informes button:hover {

5.14 Tabla del inventario

```
433  .tabla-inventario {
434
435      width: 100%;
436
437      border-collapse: collapse;
438
439      margin-top: 20px;
440  }
441
442  .tabla-inventario th,
443  .tabla-inventario td {
444
445      border: 2px solid white;
446
447      padding: 10px;
448
449      text-align: center;
450  }
451
452  .tabla-inventario th {
453
454      background-color: var(--azul);
455  }
```

Explicación técnica

La clase .tabla-inventario da formato a la tabla generada dinámicamente desde JavaScript.

.tabla-inventario {

border-collapse elimina espacios entre bordes de la tabla.

border-collapse: collapse;

Las celdas utilizan bordes blancos y alineación centrada.

text-align: center;

Los encabezados th utilizan fondo azul para destacar.

```
.tabla-inventario th {
```

6. Documentación Técnica de login.js

El archivo login.js contiene la lógica JavaScript encargada de gestionar el sistema básico de inicio de sesión y creación de cuentas de la página web. Este script trabaja directamente con los elementos HTML del formulario de login y utiliza eventos para interactuar con el usuario.

6.1 Array de usuarios

```
2  const usuarios = [];  
3  
4  
5  // ELEMENTOS HTML  
6  const formulario = document.getElementById("formularioInicioSesion");  
7  
8  const usuarioInput = document.getElementById("usuario");  
9  const passwordInput = document.getElementById("password");  
10  
11  const botonCrear = document.getElementById("crearCuenta");  
12  
13  const avisos = document.getElementById("avisos");
```

Explicación técnica

El array usuarios funciona como una base de datos temporal almacenada en memoria.

```
const usuarios = [];
```

Dentro de este array se guardarán los usuarios creados desde el formulario.

Cada usuario se almacena como un objeto con nombre y contraseña.

6.2 Obtención de elementos HTML

```
document.getElementById("usuario");
```

Explicación técnica

Se obtienen referencias a los elementos HTML mediante document.getElementById().

```
document.getElementById("usuario");
```

Esto permite manipular los elementos desde JavaScript.

Se almacenan referencias al formulario, inputs, botones y zona de avisos.

6.3 Evento de inicio de sesión

```

21      formulario.addEventListener("submit", function(e){
22
23          e.preventDefault();
24          const usuario = usuarioInput.value.trim();
25          const password = passwordInput.value.trim();
26
27          // BUSCAR USUARIO
28          const encontrado = usuarios.find(function(u){
29              return (
30                  u.usuario === usuario &&
31                  u.password === password
32              );
33          });
34          // MENSAJES
35          if(encontrado){
36              avisos.textContent =
37                  "Inicio de sesión exitoso";
38          }
39          else{
40              avisos.textContent =
41                  "La cuenta no existe";
42          }
43      });

```

Explicación técnica

Se añade un evento submit al formulario de inicio de sesión.

```
formulario.addEventListener("submit", function(e){
```

El método preventDefault() evita que la página se recargue automáticamente.

```
e.preventDefault();
```

Se obtienen los valores escritos por el usuario utilizando value.

```
usuarioInput.value
```

trim() elimina espacios vacíos al principio y final del texto.

```
trim();
```

6.4 Búsqueda de usuario

```

27      // BUSCAR USUARIO
28      const encontrado = usuarios.find(function(u){
29          return (
30              u.usuario === usuario &&
31              u.password === password
32          );
33      });

```

Explicación técnica

El método find() busca un usuario dentro del array usuarios.

```
usuarios.find(function(u){
```

La búsqueda compara usuario y contraseña.

```
u.usuario === usuario
```

Si encuentra coincidencia, devuelve el objeto del usuario.

Si no encuentra coincidencias, devuelve undefined.

6.5 Mensajes de inicio de sesión

```
35         if(encontrado){
36             avisos.textContent =
37                 "Inicio de sesión exitoso";
38         }
39         else{
40             avisos.textContent =
41                 "La cuenta no existe";
42         }
43     });
```

Explicación técnica

La estructura if comprueba si el usuario ha sido encontrado.

```
if(encontrado){
```

Si existe, se muestra un mensaje de inicio de sesión correcto.

```
"Inicio de sesión exitoso";
```

Si no existe, se informa de que la cuenta no existe.

```
"La cuenta no existe";
```

Los mensajes se muestran modificando textContent del div avisos.

```
avisos.textContent
```

6.6 Evento de creación de cuenta

```
49     botonCrear.addEventListener("click", function(){
50
51         const usuario = usuarioInput.value.trim();
52         const password = passwordInput.value.trim();
```

Explicación técnica

Se añade un evento click al botón Crear cuenta.

```
botonCrear.addEventListener("click", function(){
```

Cuando el usuario pulsa el botón, se ejecuta la función de registro.

Se vuelven a obtener los datos escritos en los inputs.

6.7 Validaciones del formulario

```
55     if(usuario === "" || password === ""){
56         avisos.textContent =
57         |     "Todos los campos son obligatorios";
58         return;
59     }
60     if(usuario.length < 4){
61         avisos.textContent =
62         |     "El usuario debe tener mínimo 4 caracteres";
63         return;
64     }
65     if(password.length < 4){
66         avisos.textContent =
67         |     "La contraseña debe tener mínimo 4 caracteres";
68         return;
69     }
```

Explicación técnica

Se comprueba que los campos no estén vacíos.

```
if(usuario === "" || password === ""){
```

También se valida la longitud mínima del usuario.

```
usuario.length < 4
```

La contraseña también debe tener al menos 4 caracteres.

```
password.length < 4
```

Si alguna validación falla, se muestra un mensaje y se detiene la función mediante return.

```
return;
```

6.8 Comprobación de usuarios existentes

```

72     const existe = usuarios.find(function(u){
73         |     return u.usuario === usuario;
74     });
75     if(existe){
76         |     avisos.textContent =
77         |         "Ese usuario ya existe";
78         return;
79     }

```

Explicación técnica

Antes de crear una cuenta, se comprueba si el usuario ya existe.

```
const existe = usuarios.find(function(u){
```

La búsqueda compara únicamente el nombre de usuario.

```
u.usuario === usuario
```

Si el usuario ya existe, se muestra un mensaje de error.

```
"Ese usuario ya existe";
```

6.9 Creación de usuario

```

82     usuarios.push({
83         |     usuario: usuario,
84         |     password: password
85     });
86     avisos.textContent =
87     |     "Cuenta creada correctamente";
88     console.log(usuarios);
89 });

```

Explicación técnica

El método push() añade un nuevo objeto al array usuarios.

```
usuarios.push({
```

El objeto almacena usuario y contraseña.

```
password: password
```

Tras crear la cuenta, se muestra un mensaje de confirmación.

```
"Cuenta creada correctamente";
```

console.log(usuarios) muestra el contenido del array en la consola del navegador.

```
console.log(usuarios);
```

7. Documentación Técnica de javas.js

El archivo `javas.js` contiene la lógica principal del mapa interactivo de la página web. Este script se encarga de generar la estructura visual del taller, mostrar armarios, baldas y cajones, buscar materiales concretos y actualizar dinámicamente la zona de información sin recargar la página.

Para documentarlo correctamente, se divide en dos partes principales: primero las estructuras de datos que almacenan la información del taller y después cada función JavaScript de forma individual.

7.1 Objeto estructura

```

1  const estructura = [
2      GENERAL: {
3          nombre: "Vista general",
4          baldas: {
5              B001: [],
6              B002: ["C00201", "C00202", "C00203", "C00204", "C00205", "C00206"],
7              B003: ["C00301"],
8              B004: [],
9              B005: [],
10             B006: ["C00601", "C00602", "C00603", "C00604", "C00605", "C00606"],
11             B007: ["C00701"],
12             B008: ["C00801", "C00802", "C00803", "C00804", "C00805"],
13             B009: ["C00901", "C00902"],
14             B010: ["C01001", "C01002", "C01003", "C01004"],
15             B011: [],
16             B012: [],
17             B013: ["C01301", "C01302"],
18             B014: ["C01401", "C01402"],
19             B015: [],
20             B016: ["C01601", "C01602", "C01603", "C01604", "C01605"],
21             B017: ["C01701"],
22             B018: ["C01801", "C01802"]
23         }
24     },
25
26     A1: {
27         nombre: "Armario A1",
28         baldas: {
29             B101: [],
30             B102: [],
31             B103: [],
32             B104: [],
33             B105: [],
34             B106: [],
35             B107: [],
36             B108: [],
37             B109: [],
38             B110: [],
39             B111: [],
40             B112: []
41         }
42     },
43
44     A2: {
45         nombre: "Armario A2",
46         baldas: {
47             B201: [],
48             B202: []
49         }
50     },
51
52     A3: {
53         nombre: "Armario A3",
54         baldas: {
55             B301: [],
56             B302: []
57         }
58     },
59
60     A4: {
61         nombre: "Armario A4",
62         baldas: {
63             B401: [],
64             B402: []
65         }
66     },
67
68     A5: {
69         nombre: "Armario A5",
70         baldas: {
71             B501: [],
72             B502: [],
73             B503: [],
74             B504: [],
75             B505: [],
76             B506: [],
77             B507: [],
78             B508: [],
79             B509: [],
80             B510: [],
81             B511: [],
82             B512: []
83         }
84     }
85 ]

```


Explicación técnica

El objeto estructura funciona como el modelo interno de la distribución física del taller. En él se definen las zonas principales, como GENERAL, A1, A2, A3, A4 y A5.

```
const estructura = {
```

Cada armario contiene una propiedad nombre, que sirve para identificarlo de forma descriptiva.

```
  nombre: "Vista general"
```

La propiedad baldas almacena las baldas disponibles dentro de cada armario.

```
  baldas: {
```

Cada balda se identifica mediante un código, por ejemplo B002, B010 o B507.

```
    B002: ["C00201", "C00202", "C00203", "C00204", "C00205", "C00206"]
```

Cuando una balda tiene cajones, estos se guardan dentro de un array. Si no tiene cajones, el array aparece vacío.

```
    B001: []
```

Esta estructura permite que el programa sepa qué debe dibujar en el mapa cuando el usuario abre un armario o una balda.

7.2 Array materiales

```

87  const materiales = [
88      {
89          nombre: "Arduino UNO",
90          cantidad: 5,
91          armario: "GENERAL",
92          balda: "B002",
93          cajon: "C00201",
94          descripcion: "Placas Arduino para pr cticas"
95      },
96      {
97          nombre: "Cable HDMI",
98          cantidad: 10,
99          armario: "GENERAL",
100         balda: "B010",
101         cajon: "C01001",
102         descripcion: "Cables HDMI para monitores"
103     },
104     {
105         nombre: "Raspberry Pi",
106         cantidad: 3,
107         armario: "GENERAL",
108         balda: "B003",
109         cajon: "C00301",
110         descripcion: "Miniordenadores Raspberry"
111     },
112     {
113         nombre: "Protoboard",
114         cantidad: 8,
115         armario: "GENERAL",
116         balda: "B006",
117         cajon: "C00601",
118         descripcion: "Placas de pruebas electr nicas"
119     },
120     {
121         nombre: "Resistencias",
122         cantidad: 100,
123         armario: "GENERAL",
124         balda: "B008",
125         cajon: "C00801",
126         descripcion: "Resistencias varias"
127     },
128     {
129         nombre: "Cable VGA",
130         cantidad: 7,
131         armario: "GENERAL",
132         balda: "B009",
133         cajon: "C00901",
134         descripcion: "Cables VGA para monitores antiguos"
135     },
136     {
137         nombre: "Fuente ATX",
138         cantidad: 4,
139         armario: "A1",
140         balda: "B103",
141         cajon: "",
142         descripcion: "Fuentes de alimentaci n ATX"
143     },
144     {
145         nombre: "ESP32",
146         cantidad: 12,
147         armario: "GENERAL",
148         balda: "B016",
149         cajon: "C01603",
150         descripcion: "Microcontroladores ESP32 con WiFi"
151     },
152     {
153         nombre: "Switch 24 Puertos",
154         cantidad: 2,
155         armario: "A5",
156         balda: "B507",
157         cajon: "",
158         descripcion: "Switches de red para pr cticas"
159     },
160     {
161         nombre: "LED Azul",
162         cantidad: 60,
163         armario: "GENERAL",
164         balda: "B008",
165         cajon: "C00803",
166         descripcion: "LEDs azules de 5mm"
167     },
168 ];

```

Explicación técnica

El array materiales actúa como una pequeña base de datos interna del inventario.

```
const materiales = [
```

Cada elemento del array es un objeto que representa un material concreto.

```
{ nombre: "Arduino UNO", cantidad: 5, armario: "GENERAL", balda: "B002", cajon: "C00201" }
```

El campo nombre guarda el nombre visible del componente.

```
nombre: "Arduino UNO"
```

El campo cantidad indica cuántas unidades hay disponibles.

```
cantidad: 5
```

Los campos armario, balda y cajon indican la ubicación exacta del material.

```
armario: "GENERAL", balda: "B002", cajon: "C00201"
```

El campo descripcion aporta información adicional que luego se muestra en la zona de información.

```
descripcion: "Placas Arduino para prácticas"
```

Si el campo cajon está vacío, significa que el material está ubicado directamente en una balda y no dentro de un cajón concreto.

```
cajon: ""
```

7.3 Función verMapaGeneral()

```

170 function verMapaGeneral() {
171     document.getElementById("tituloMapa").innerHTML = "Vista general del armario";
172
173     document.getElementById("mapaDinamico").innerHTML = `
174         <div class="armario-general">
175
176             <div class="fila">
177                 <div id="B001" class="zona balda-larga" onclick="verBalda('GENERAL', 'B001')">B001</div>
178             </div>
179
180             <div class="fila">
181                 <div id="B002" class="zona balda" onclick="verBalda('GENERAL', 'B002')">B002</div>
182                 <div id="B003" class="zona balda" onclick="verBalda('GENERAL', 'B003')">B003</div>
183                 <div id="B004" class="zona balda" onclick="verBalda('GENERAL', 'B004')">B004</div>
184                 <div id="B005" class="zona balda" onclick="verBalda('GENERAL', 'B005')">B005</div>
185             </div>
186
187             <div class="fila">
188                 <div id="B006" class="zona balda" onclick="verBalda('GENERAL', 'B006')">B006</div>
189                 <div id="B007" class="zona balda" onclick="verBalda('GENERAL', 'B007')">B007</div>
190                 <div id="B008" class="zona balda" onclick="verBalda('GENERAL', 'B008')">B008</div>
191                 <div id="B009" class="zona balda" onclick="verBalda('GENERAL', 'B009')">B009</div>
192             </div>
193
194             <div class="fila">
195                 <div id="A1" class="zona armario-alto" onclick="verArmario('A1')">A1</div>
196
197                 <div style="flex: 3;">
198                     <div class="fila">
199                         <div id="B010" class="zona balda" onclick="verBalda('GENERAL', 'B010')">B010</div>
200                         <div id="B013" class="zona balda" onclick="verBalda('GENERAL', 'B013')">B013</div>
201                         <div id="B016" class="zona balda" onclick="verBalda('GENERAL', 'B016')">B016</div>
202                     </div>
203
204                     <div class="fila">
205                         <div id="B011" class="zona balda" onclick="verBalda('GENERAL', 'B011')">B011</div>
206                         <div id="B014" class="zona balda" onclick="verBalda('GENERAL', 'B014')">B014</div>
207                         <div id="B017" class="zona balda" onclick="verBalda('GENERAL', 'B017')">B017</div>
208                     </div>
209
210                     <div class="fila">
211                         <div id="B012" class="zona balda" onclick="verBalda('GENERAL', 'B012')">B012</div>
212                         <div id="B015" class="zona balda" onclick="verBalda('GENERAL', 'B015')">B015</div>
213                         <div id="B018" class="zona balda" onclick="verBalda('GENERAL', 'B018')">B018</div>
214                     </div>
215
216                     <div class="fila">
217                         <div id="A2" class="zona armario" onclick="verArmario('A2')">A2</div>
218                         <div id="A3" class="zona armario" onclick="verArmario('A3')">A3</div>
219                         <div id="A4" class="zona armario" onclick="verArmario('A4')">A4</div>
220                     </div>
221                 </div>
222
223                 <div id="A5" class="zona armario-alto" onclick="verArmario('A5')">A5</div>
224             </div>
225         </div>
226     `;
227 }
228

```

Explicación técnica

Esta función genera la vista principal del mapa del taller.

```
function verMapaGeneral() {
```

Primero modifica el título del mapa para indicar que se está viendo la vista general.

```
document.getElementById("tituloMapa").innerHTML = "Vista general del armario";
```

Después accede al contenedor mapaDinamico y sustituye su contenido mediante innerHTML.

```
document.getElementById("mapaDinamico").innerHTML = `
```

Dentro de ese innerHTML se escribe una plantilla HTML con armarios y baldas.

```
<div class="armario-general">
```

Cada zona del mapa tiene un id único, una clase CSS y un evento onclick.

```
<div id="B002" class="zona balda" onclick="verBalda('GENERAL', 'B002')">B002</div>
```

Cuando el usuario pulsa una balda, se ejecuta verBalda(). Cuando pulsa un armario, se ejecuta verArmario().

Esta función es la que se carga al abrir la página por primera vez.

7.4 Función verArmario(codigoArmario)

```
230 function verArmario(codigoArmario) {
231   let armario = estructura[codigoArmario];
232   let baldas = Object.keys(armario.baldas);
233
234   document.getElementById("tituloMapa").innerHTML = `Interior de ${codigoArmario}`;
235
236   let html = `
237     <div class="interior-armario">
238       <div class="ruta">
239         <button onclick="verMapaGeneral()"> Volver al mapa general</button>
240         ${codigoArmario}
241       </div>
242     `;
243
244   for (let i = 0; i < baldas.length; i += 2) {
245     html += `<div class="fila">`;
246
247     html += `
248       <div id="${baldas[i]}" class="zona balda" onclick="verBalda('${codigoArmario}', '${baldas[i]}')">
249         |
250         ${baldas[i]}
251       </div>
252     `;
253
254     if (baldas[i + 1] !== undefined) {
255       html += `
256       <div id="${baldas[i + 1]}" class="zona balda" onclick="verBalda('${codigoArmario}', '${baldas[i + 1]}')">
257         |
258         ${baldas[i + 1]}
259       </div>
260     `;
261     html += `</div>`;
262
263     html += `</div>`;
264
265     document.getElementById("mapaDinamico").innerHTML = html;
266     mostrarMaterialesArmario(codigoArmario);
267   }
268 }
```

Explicación técnica

Esta función muestra el interior de un armario concreto.

```
function verArmario(codigoArmario) {
```

Recibe como parámetro el código del armario, por ejemplo A1 o A5.

```
codigoArmario
```

Busca dentro del objeto estructura el armario indicado.

```
let armario = estructura[codigoArmario];
```

Object.keys obtiene todos los nombres de las baldas de ese armario.

```
let baldas = Object.keys(armario.baldas);
```

Después cambia el título del mapa para indicar qué armario se está viendo.

```
document.getElementById("tituloMapa").innerHTML = `Interior de ${codigoArmario}`;
```

La variable html se utiliza para construir el contenido que se insertará en la página.

```
let html = `
```

El bucle for recorre las baldas de dos en dos para colocarlas en filas.

```
for (let i = 0; i < baldas.length; i += 2) {
```

Si existe una segunda balda en la misma fila, también la añade al HTML.

```
if (baldas[i + 1] !== undefined) {
```

Al final, el HTML generado se inserta dentro de mapaDinamico.

```
document.getElementById("mapaDinamico").innerHTML = html;
```

Por último, se llama a mostrarMaterialesArmario() para listar los materiales de ese armario.

```
mostrarMaterialesArmario(codigoArmario);
```

7.5 Función verBalda(codigoArmario, codigoBalda)

```
271 function verBalda(codigoArmario, codigoBalda) {
272     let cajones = estructura[codigoArmario].baldas[codigoBalda];
273
274     document.getElementById("tituloMapa").innerHTML = `Interior de ${codigoBalda}`;
275
276     let html = `
277         <div class="interior-armario">
278             <div class="ruta">
279                 <p><strong>${codigoArmario} > ${codigoBalda}</strong></p>
280                 <button onclick="verMapaGeneral()"> Mapa general</button>
281             </div>
282         `;
283
284         if (cajones.length > 0) {
285             for (let i = 0; i < cajones.length; i += 2) {
286                 html += `<div class="fila">`;
287
288                 html += `
289                     <div id="${cajones[i]}" class="zona cajon" onclick="verCajon('${codigoArmario}', '${codigoBalda}')">
290                         ${cajones[i]}
291                     </div>
292                 `;
293
294                 if (cajones[i + 1] !== undefined) {
295                     html += `
296                         <div id="${cajones[i + 1]}" class="zona cajon" onclick="verCajon('${codigoArmario}', '${codigoBalda}')">
297                             ${cajones[i + 1]}
298                         </div>
299                     `;
300                 }
301
302                 html += `</div>`;
303             }
304         }
305
306         html += `</div>`;
307
308         document.getElementById("mapaDinamico").innerHTML = html;
309
310         mostrarMaterialesBalda(codigoArmario, codigoBalda);
311     }
```

Explicación técnica

Esta función muestra el interior de una balda concreta.

```
function verBalda(codigoArmario, codigoBalda) {
```

Recibe dos parámetros: el armario donde se encuentra la balda y el código de la balda.

```
codigoArmario, codigoBalda
```

Obtiene los cajones asociados a esa balda desde el objeto estructura.

```
let cajones = estructura[codigoArmario].baldas[codigoBalda];
```

Actualiza el título del mapa para indicar la balda que se está visualizando.

```
document.getElementById("tituloMapa").innerHTML = `Interior de ${codigoBalda}`;
```

Crea una ruta visual que muestra la relación armario > balda.

```
<p><strong>${codigoArmario} > ${codigoBalda}</strong></p>
```

Si la balda tiene cajones, los recorre con un bucle for.

```
if (cajones.length > 0) {
```

El bucle avanza de dos en dos para colocar los cajones en filas.

```
for (let i = 0; i < cajones.length; i += 2) {
```

Cada cajón generado tiene un onclick que llama a verCajon().

```
onclick="verCajon('${codigoArmario}', '${codigoBalda}', '${cajones[i]}')"
```

Finalmente, muestra los materiales que pertenecen a esa balda.

```
mostrarMaterialesBalda(codigoArmario, codigoBalda);
```

7.6 Función verCajon(codigoArmario, codigoBalda, codigoCajon)

```
313 function verCajon(codigoArmario, codigoBalda, codigoCajon) {
314
315     marcarZona(codigoCajon);
316
317     let encontrados = materiales.filter(m =>
318         m.armario === codigoArmario &&
319         m.balda === codigoBalda &&
320         m.cajon === codigoCajon
321     );
322
323     document.getElementById("info").innerHTML = `
324         <h3>${codigoArmario} > ${codigoBalda} > ${codigoCajon}</h3>
325         ${crearTextoMateriales(encontrados)}
326     `;
327 }
```

Explicación técnica

Esta función se ejecuta cuando el usuario pulsa sobre un cajón.

```
function verCajon(codigoArmario, codigoBalda, codigoCajon) {
```

Primero marca visualmente el cajón seleccionado.

```
marcarZona(codigoCajon);
```

Después filtra el array materiales para encontrar solo los materiales que coinciden con ese armario, esa balda y ese cajón.

```
let encontrados = materiales.filter(m =>
```

La condición utiliza comparaciones estrictas para comprobar la ubicación exacta.

```
m.armario === codigoArmario && m.balda === codigoBalda && m.cajon === codigoCajon
```

Finalmente actualiza la caja info mostrando la ruta seleccionada y los materiales encontrados.

```
document.getElementById("info").innerHTML = `
```

Para convertir la lista de materiales en HTML utiliza la función crearTextoMateriales().

```
${crearTextoMateriales(encontrados)}
```

7.7 Función limpiarMarcado()

```
329 function limpiarMarcado() {
330     let zonas = document.getElementsByClassName("zona");
331
332     for (let i = 0; i < zonas.length; i++) {
333         zonas[i].classList.remove("zona-activa");
334     }
335 }
```

Explicación técnica

Esta función elimina cualquier selección visual anterior del mapa.

```
function limpiarMarcado() {
```

Primero obtiene todos los elementos que tienen la clase zona.

```
let zonas = document.getElementsByClassName("zona");
```

Después recorre todos esos elementos con un bucle for.

```
for (let i = 0; i < zonas.length; i++) {
```

A cada zona le elimina la clase zona-activa.

```
zonas[i].classList.remove("zona-activa");
```

Esto evita que queden varias zonas resaltadas al mismo tiempo.

7.8 Función marcarZona(codigo)


```

337 function marcarZona(codigo) {
338     limpiarMarcado();
339
340     let zona = document.getElementById(codigo);
341
342     if (zona !== null) {
343         zona.classList.add("zona-activa");
344     }
345 }

```

Explicación técnica

Esta función resalta visualmente una zona concreta del mapa.

`function marcarZona(codigo) {`

Primero llama a `limpiarMarcado()` para borrar selecciones anteriores.

`limpiarMarcado();`

Después busca el elemento HTML cuyo id coincida con el código recibido.

`let zona = document.getElementById(codigo);`

Comprueba que el elemento exista antes de modificarlo.

`if (zona !== null) {`

Si existe, añade la clase CSS zona-activa.

`zona.classList.add("zona-activa");`

La clase zona-activa cambia el color y borde del elemento seleccionado.

7.9 Función `buscarMaterial(nombre)`

```

347 function buscarMaterial(nombre) {
348
349     let material = materiales.find(m => m.nombre === nombre);
350
351     if (material === undefined) {
352
353         verMapaGeneral();
354         limpiarMarcado();
355
356         document.getElementById("info").innerHTML = `
357         <h3>${nombre}</h3>
358         <p>Este componente todavía no tiene ubicación registrada.</p>
359         `;
360
361         return;
362     }
363
364     if (material.cajon !== "") {
365
366         verBalda(material.armario, material.balda);
367
368         setTimeout(() => {
369             marcarZona(material.cajon);
370         }, 50);
371
372     } else {
373
374         if (material.armario === "GENERAL") {
375             verMapaGeneral();
376         } else {
377             verArmario(material.armario);
378         }
379
380         setTimeout(() => {
381             marcarZona(material.balda);
382         }, 50);
383     }
384
385     setTimeout(() => {
386
387         document.getElementById("info").innerHTML = `
388         <h3>${material.nombre}</h3>
389
390         <p><strong>Cantidad:</strong> ${material.cantidad}</p>
391
392         <p><strong>Ubicación:</strong>
393         <div>
394             ${material.armario}
395             </div>
396             <div>
397                 ${material.balda}
398             </div>
399             ${material.cajon !== "" ? "> " + material.cajon : ""}
400         </p>
401
402         <p><strong>Descripción:</strong> ${material.descripcion}</p>
403         `;
404
405         }, 60);
406     }
407 }

```

Explicación técnica

Esta es una de las funciones más importantes del archivo, porque permite localizar automáticamente un componente.

function buscarMaterial(nombre) {

Recibe como parámetro el nombre del material pulsado en el menú lateral.

`nombre`

Busca ese material dentro del array materiales utilizando find().

```
let material = materiales.find(m => m.nombre === nombre);
```

Si no encuentra el material, material será undefined.

```
if (material === undefined) {
```

En ese caso vuelve al mapa general, limpia el marcado y muestra un mensaje informativo.

```
verMapaGeneral(); limpiarMarcado();
```

El return termina la función para evitar que siga ejecutándose.

```
return;
```

Si el material tiene cajón, abre directamente la balda correspondiente.

```
if (material.cajon !== "") { verBalda(material.armario, material.balda); }
```

Después usa setTimeout para esperar unos milisegundos antes de marcar el cajón.

```
setTimeout(() => { marcarZona(material.cajon); }, 50);
```

El setTimeout es necesario porque primero se genera el HTML dinámico y después se puede seleccionar el elemento.

Si el material no tiene cajón, el sistema abre el armario o la vista general y marca la balda.

```
marcarZona(material.balda);
```

Finalmente se actualiza la caja info con nombre, cantidad, ubicación y descripción del material.

```
document.getElementById("info").innerHTML = `
```

7.10 Función mostrarMaterialesArmario(codigoArmario)

```
409 function mostrarMaterialesArmario(codigoArmario) {  
410     let encontrados = materiales.filter(m => m.armario === codigoArmario);  
411  
412     document.getElementById("info").innerHTML = `  
413         <h3>Contenido general de ${codigoArmario}</h3>  
414         ${crearTextoMateriales(encontrados)}  
415     `;  
416 }
```

Explicación técnica

Esta función muestra todos los materiales que pertenecen a un armario concreto.

```
function mostrarMaterialesArmario(codigoArmario) {
```

Utiliza filter() para seleccionar únicamente los materiales cuyo campo armario coincide con el parámetro recibido.

```
let encontrados = materiales.filter(m => m.armario === codigoArmario);
```

Después actualiza la caja info con un título y la lista de materiales.

```
document.getElementById("info").innerHTML = `
```

La función `crearTextoMateriales()` se encarga de transformar los objetos del array en código HTML visible.

```
`${crearTextoMateriales(encontrados)}`
```

7.11 Función `mostrarMaterialesBalda(codigoArmario, codigoBalda)`

```
418 function mostrarMaterialesBalda(codigoArmario, codigoBalda) {
419   let encontrados = materiales.filter(m =>
420     m.armario === codigoArmario &&
421     m.balda === codigoBalda
422   );
423
424   document.getElementById("info").innerHTML = `
425     <h3>Contenido de ${codigoArmario} > ${codigoBalda}</h3>
426     ${crearTextoMateriales(encontrados)}
427   `;
428 }
```

Explicación técnica

Esta función muestra todos los materiales ubicados en una balda concreta.

```
function mostrarMaterialesBalda(codigoArmario, codigoBalda) {
```

Filtra el array `materiales` comprobando dos condiciones: `armario` y `balda`.

```
let encontrados = materiales.filter(m => m.armario === codigoArmario && m.balda ===
codigoBalda);
```

Después escribe el resultado en la caja `info`.

```
document.getElementById("info").innerHTML = `
```

La ruta mostrada permite saber exactamente qué zona del mapa se está consultando.

```
<h3>Contenido de ${codigoArmario} > ${codigoBalda}</h3>
```

7.12 Función `crearTextoMateriales(lista)`

```

430 function crearTextoMateriales(lista) {
431     if (lista.length === 0) {
432         return `<p>No hay materiales registrados aquí.</p>`;
433     }
434
435     let texto = "";
436
437     lista.forEach(m => {
438         texto += `
439             <div>
440                 <p>
441                     <strong>${m.nombre}</strong><br>
442                     Cantidad: ${m.cantidad}<br>
443                     Ubicación: ${m.armario} > ${m.balda} > ${m.cajon}<br>
444                     ${m.descripcion}
445                 </p>
446                 <hr>
447             </div>
448         `;
449     });
450
451     return texto;
452 }
453
454 verMapaGeneral();

```

Explicación técnica

Esta función recibe una lista de materiales y la transforma en texto HTML.

```
function crearTextoMateriales(lista) {
```

Si la lista está vacía, devuelve un mensaje indicando que no hay materiales registrados.

```
if (lista.length === 0) { return `<p>No hay materiales registrados aquí.</p>`; }
```

Si hay materiales, crea una variable texto inicialmente vacía.

```
let texto = "";
```

Después recorre la lista con `forEach()`.

```
lista.forEach(m => {
```

Por cada material añade un bloque HTML con nombre, cantidad, ubicación y descripción.

```
<strong>${m.nombre}</strong><br>
```

El separador `hr` se utiliza para distinguir visualmente un material de otro.

```
<hr>
```

Finalmente devuelve todo el texto HTML generado.

```
return texto;
```

7.13 Llamada inicial `verMapaGeneral()`

```
454 verMapaGeneral();
```

Explicación técnica

Al final del archivo se ejecuta la función `verMapaGeneral()`.

`verMapaGeneral();`

Esta llamada hace que, al cargar la página Mapa.html, el mapa general aparezca automáticamente sin que el usuario tenga que pulsar ningún botón.

Es el punto de inicio visual del sistema interactivo.

7.14 Resumen del funcionamiento general

El archivo `javas.js` combina estructuras de datos y funciones para crear un mapa interactivo. El objeto `estructura` define la organización física del taller, mientras que el array `materiales` define qué componentes existen y dónde se encuentran.

Las funciones `verMapaGeneral()`, `verArmario()`, `verBalda()` y `verCajon()` se encargan de navegar por el mapa. Las funciones `limpiarMarcado()` y `marcarZona()` gestionan el resaltado visual. Las funciones `mostrarMaterialesArmario()`, `mostrarMaterialesBalda()` y `crearTextoMateriales()` generan la información que se muestra al usuario. Por último, `buscarMaterial()` une todas estas partes para localizar automáticamente un componente desde el menú lateral.

8. Documentación Técnica de informes.js

El archivo `informes.js` contiene la lógica encargada de leer el archivo `inventario.xml` y convertir sus datos en dos tipos de informe: una tabla HTML visible dentro de la página `Inicio.html` y un archivo CSV descargable. Este script permite que la información del inventario no esté escrita directamente en el HTML, sino que pueda obtenerse desde un archivo XML externo.

8.1 Función `cargarXML(callback)`

```
1 function cargarXML(callback) {  
2     fetch("inventario.xml")  
3     .then(respuesta => respuesta.text())  
4     .then(texto => {  
5         let parser = new DOMParser();  
6         let xml = parser.parseFromString(texto, "text/xml");  
7         callback(xml);  
8     });  
9 }
```

Explicación técnica

Esta función es la base del archivo, ya que se encarga de cargar el documento `inventario.xml` para que el resto de funciones puedan trabajar con sus datos.

`function cargarXML(callback) {`

La instrucción `fetch` realiza una petición al archivo `inventario.xml`. En este caso, el archivo se encuentra en la misma carpeta que la página web.

`fetch("inventario.xml")`

Después, la respuesta recibida se convierte a texto mediante `respuesta.text()`. Esto es necesario porque el XML primero llega como contenido textual.

`.then(respuesta => respuesta.text())`

El objeto DOMParser permite transformar ese texto en un documento XML real que JavaScript puede recorrer y consultar.

```
let parser = new DOMParser();
```

parseFromString convierte el texto recibido en un documento XML utilizando el tipo text/xml.

```
let xml = parser.parseFromString(texto, "text/xml");
```

Finalmente, callback(xml) devuelve el XML ya procesado a la función que llamó a cargarXML(). Gracias a esto, cargarXML puede reutilizarse tanto para generar CSV como para mostrar la tabla HTML.

```
callback(xml);
```

8.2 Función obtenerTexto(material, etiqueta)

```
11 function obtenerTexto(material, etiqueta) {  
12   |   return material.getElementsByTagName(etiqueta)[0].textContent;  
13 }
```

Explicación técnica

Esta función auxiliar sirve para obtener el contenido textual de una etiqueta concreta dentro de un material del XML.

```
function obtenerTexto(material, etiqueta) {
```

El parámetro material representa un nodo <material> del XML.

```
material
```

El parámetro etiqueta representa el nombre de la etiqueta que se quiere consultar, por ejemplo nombre, cantidad, armario, balda, cajon o descripcion.

```
etiqueta
```

getElementsByTagName(etiqueta)[0] busca la primera etiqueta con ese nombre dentro del material.

```
material.getElementsByTagName(etiqueta)[0]
```

textContent extrae únicamente el texto que hay dentro de la etiqueta.

```
.textContent
```

Esta función evita repetir varias veces el mismo código cuando se generan el CSV y la tabla HTML.

8.3 Función generarCSVDesdeXML()

```

15 function generarCSVDesdeXML() {
16     cargarXML(function(xml) {
17         let materiales = xml.getElementsByTagName("material");
18
19         let csv = "Nombre;Cantidad;Armario;Balda;Cajon;Descripcion\n";
20     });

```

Explicación técnica

Esta función genera un archivo CSV descargable a partir de los datos almacenados en inventario.xml.

```
function generarCSVDesdeXML() {
```

Primero llama a cargarXML() para leer el archivo XML. El código que genera el CSV se ejecuta dentro del callback, cuando el XML ya está cargado.

```
cargarXML(function(xml) {
```

getElementsByTagName("material") obtiene todos los elementos <material> del XML.

```
let materiales = xml.getElementsByTagName("material");
```

La variable csv empieza con una primera línea que actúa como cabecera del archivo. Los campos se separan por punto y coma.

```
let csv = "Nombre;Cantidad;Armario;Balda;Cajon;Descripcion\n";
```

El salto de línea

permite que cada material aparezca en una fila diferente dentro del archivo CSV.

```
\n
```

8.4 Recorrido de materiales y construcción del CSV

```

21 for (let i = 0; i < materiales.length; i++) {
22     let m = materiales[i];
23
24     csv += obtenerTexto(m, "nombre") + ";";
25     csv += obtenerTexto(m, "cantidad") + ";";
26     csv += obtenerTexto(m, "armario") + ";";
27     csv += obtenerTexto(m, "balda") + ";";
28     csv += obtenerTexto(m, "cajon") + ";";
29     csv += obtenerTexto(m, "descripcion") + "\n";
30 }

```

Explicación técnica

El bucle for recorre todos los materiales encontrados en el XML.

```
for (let i = 0; i < materiales.length; i++) {
```

En cada vuelta del bucle, la variable m representa un material concreto.

```
let m = materiales[i];
```

Por cada material se van añadiendo sus campos al texto CSV.


```
csv += obtenerTexto(m, "nombre") + ",";
```

La función `obtenerTexto()` se reutiliza para extraer el nombre, cantidad, armario, balda, cajón y descripción.

```
obtenerTexto(m, "descripcion")
```

Al final de cada material se añade un salto de línea para pasar a la siguiente fila.

```
+ "\n";
```

8.5 Creación y descarga del archivo CSV

```
32     let archivo = new Blob([csv], { type: "text/csv" });
33     let enlace = document.createElement("a");
34
35     enlace.href = URL.createObjectURL(archivo);
36     enlace.download = "informe_inventario.csv";
37     enlace.click();
38   });
39 }
```

Explicación técnica

Una vez generado el texto CSV, se crea un archivo temporal en memoria usando `Blob`.

```
let archivo = new Blob([csv], { type: "text/csv" });
```

`Blob` permite construir archivos desde JavaScript sin necesidad de un servidor.

```
new Blob([csv], { type: "text/csv" })
```

Después se crea dinámicamente un enlace HTML con `document.createElement("a")`.

```
let enlace = document.createElement("a");
```

`URL.createObjectURL(archivo)` genera una URL temporal que apunta al archivo creado en memoria.

```
enlace.href = URL.createObjectURL(archivo);
```

La propiedad `download` define el nombre con el que se descargará el archivo.

```
enlace.download = "informe_inventario.csv";
```

Finalmente, `enlace.click()` simula un clic automático para iniciar la descarga del CSV.

```
enlace.click();
```

8.6 Función `mostrarInventarioDesdeXML()`

```

41  function mostrarInventarioDesdeXML() {
42      cargarXML(function(xml) {
43          let materiales = xml.getElementsByTagName("material");
44
45          let html = `
46              <table class="tabla-inventario">
47                  <tr>
48                      <th>Nombre</th>
49                      <th>Cantidad</th>
50                      <th>Armario</th>
51                      <th>Balda</th>
52                      <th>Cajón</th>
53                      <th>Descripción</th>
54                  </tr>
55          `;

```

Explicación técnica

Esta función lee el XML y genera una tabla HTML visible dentro de la página Inicio.html.

```
function mostrarInventarioDesdeXML() {
```

Igual que en la función anterior, primero se llama a cargarXML() para obtener el documento XML.

```
cargarXML(function(xml) {
```

Después se obtienen todos los materiales del XML.

```
let materiales = xml.getElementsByTagName("material");
```

La variable html almacena el código de la tabla que se va a insertar en la página.

```
let html = `
```

La tabla utiliza la clase tabla-inventario, definida en Estilo.css, para aplicar bordes, alineación y colores.

```
<table class="tabla-inventario">
```

La primera fila contiene los encabezados de la tabla: Nombre, Cantidad, Armario, Balda, Cajón y Descripción.

```
<th>Nombre</th>
```

8.7 Generación de filas de la tabla HTML

```

57     for (let i = 0; i < materiales.length; i++) {
58         let m = materiales[i];
59
60         html += `
61             <tr>
62                 <td>${obtenerTexto(m, "nombre")}</td>
63                 <td>${obtenerTexto(m, "cantidad")}</td>
64                 <td>${obtenerTexto(m, "armario")}</td>
65                 <td>${obtenerTexto(m, "balda")}</td>
66                 <td>${obtenerTexto(m, "cajon")}</td>
67                 <td>${obtenerTexto(m, "descripcion")}</td>
68             </tr>
69         `;
70     }

```

Explicación técnica

El bucle recorre todos los materiales existentes en el XML.

```
for (let i = 0; i < materiales.length; i++) {
```

En cada vuelta, la variable m almacena el material actual.

```
let m = materiales[i];
```

Por cada material se añade una nueva fila <tr> a la tabla.

```
<tr>
```

Cada dato del material se coloca dentro de una celda <td>.

```
<td>${obtenerTexto(m, "nombre")}</td>
```

Se usan plantillas literales con \${} para insertar valores de JavaScript dentro del HTML.

```
${obtenerTexto(m, "cantidad")}
```

Esta técnica permite construir contenido HTML dinámico a partir de datos externos.

8.8 Inserción de la tabla en la página

```

72     html += `</table>`;
73
74     document.getElementById("inventarioHTML").innerHTML = html;
75 });
76 }

```

Explicación técnica

Después de recorrer todos los materiales, se cierra la tabla.

```
html += `</table>`;
```

El contenido HTML generado se inserta dentro del div con id inventarioHTML.

```
document.getElementById("inventarioHTML").innerHTML = html;
```

Gracias a innerHTML, la tabla aparece en pantalla sin recargar la página.

innerHTML

El contenedor inventarioHTML está definido en Inicio.html y permanece vacío hasta que se pulsa el botón correspondiente.

inventarioHTML

8.9 Función ocultarInventario()

```
78 function ocultarInventario() {  
79     document.getElementById("inventarioHTML").innerHTML = "";  
80 }
```

Explicación técnica

Esta función sirve para borrar la tabla del inventario que se ha generado previamente.

function ocultarInventario() {

Selecciona el contenedor con id inventarioHTML.

document.getElementById("inventarioHTML")

Al asignar una cadena vacía a innerHTML, se elimina todo el contenido que había dentro del contenedor.

.innerHTML = "";

Esto permite ocultar la tabla sin cambiar de página y sin recargar el navegador.

8.10 Relación entre informes.js, Inicio.html e inventario.xml

Explicación técnica

El archivo informes.js no funciona de forma aislada, sino que se conecta con Inicio.html y con inventario.xml.

Inicio.html contiene los botones que ejecutan las funciones generarCSVDesdeXML(), mostrarInventarioDesdeXML() y ocultarInventario().

<button onclick="generarCSVDesdeXML()">

inventario.xml contiene los datos reales de los materiales.

<material>

informes.js actúa como intermediario: lee el XML, procesa los datos y los muestra o exporta según la acción del usuario.

9. Documentación Técnica de inventario.xml

El archivo inventario.xml funciona como el almacén de datos del inventario utilizado por la página web. En él se guardan los materiales disponibles, su cantidad, su ubicación dentro del taller y una pequeña descripción. Este documento XML es leído posteriormente por JavaScript,

concretamente por el archivo informes.js, para generar una tabla HTML y un informe CSV descargable.

A diferencia del archivo javas.js, que contiene datos internos para el mapa interactivo, inventario.xml está pensado como un fichero de datos estructurado. Esto permite que el inventario pueda leerse de forma ordenada y reutilizarse para generar informes.

9.1 Declaración inicial del documento XML

```
1 <?xml version="1.0" encoding="UTF-8"?>
```

Explicación técnica

El archivo comienza con una declaración XML. Esta línea indica que el documento está escrito en XML y especifica la codificación utilizada.

```
<?xml version="1.0" encoding="UTF-8"?>
```

El atributo version="1.0" indica la versión del estándar XML utilizada.

```
version="1.0"
```

El atributo encoding="UTF-8" permite utilizar correctamente caracteres especiales, tildes y símbolos propios del idioma español.

```
encoding="UTF-8"
```

9.2 Nodo raíz del inventario

```
2 <inventario>
```

```
92 </inventario>
```

Explicación técnica

Después de la declaración inicial aparece el nodo raíz del documento.

```
<inventario>
```

El nodo inventario actúa como contenedor principal de todos los materiales registrados.

Todo el contenido del archivo debe estar dentro de esta etiqueta para que el XML esté correctamente estructurado.

```
</inventario>
```

9.3 Bloque individual de material

```
3 <material>
4   <nombre>Arduino UNO</nombre>
5   <cantidad>5</cantidad>
6   <armario>GENERAL</armario>
7   <balda>B002</balda>
8   <cajon>C00201</cajon>
9   <descripcion>Placas Arduino para prácticas</descripcion>
10 </material>
```

Explicación técnica

Cada elemento del inventario se representa mediante una etiqueta material.

```
<material>
```

Dentro de cada bloque material se almacenan los datos concretos de un componente del taller.

La etiqueta de cierre </material> marca el final de ese componente concreto.

```
</material>
```

Gracias a esta estructura repetida, JavaScript puede recorrer todos los materiales utilizando `getElementsByTagName("material")`.

```
xml.getElementsByTagName("material")
```

9.4 Campo nombre

4		<pre><nombre>Arduino UNO</nombre></pre>
---	--	---

Explicación técnica

La etiqueta nombre guarda el nombre del componente o material.

```
<nombre>Arduino UNO</nombre>
```

Este dato se utiliza como identificador visible del material cuando se genera la tabla HTML o el archivo CSV.

En el informe, este campo aparece en la primera columna.

Nombre

9.5 Campo cantidad

5		<pre><cantidad>5</cantidad></pre>
---	--	---

Explicación técnica

La etiqueta cantidad almacena el número de unidades disponibles de un material.

```
<cantidad>5</cantidad>
```

Aunque el valor aparece escrito dentro de una etiqueta XML, representa un dato numérico.

Este campo permite saber cuántas unidades existen de cada componente.

Cantidad

9.6 Campo armario

6		<pre><armario>GENERAL</armario></pre>
---	--	---

Explicación técnica

La etiqueta armario indica en qué armario o zona principal se encuentra el material.

```
<armario>GENERAL</armario>
```

Puede contener valores como GENERAL, A1 o A5, dependiendo de la ubicación del componente.

Este campo es importante porque permite relacionar el inventario con la distribución física del taller.

Armario

9.7 Campo balda

7		<code><balda>B002</balda></code>
---	--	--

Explicación técnica

La etiqueta balda indica la balda concreta donde se encuentra el material.

`<balda>B002</balda>`

El código de la balda sigue el mismo sistema de nomenclatura utilizado en el mapa interactivo.

Este dato ayuda a localizar el componente con mayor precisión dentro del armario.

Balda

9.8 Campo cajón

8		<code><cajon>C00201</cajon></code>
---	--	--

Explicación técnica

La etiqueta cajon indica el cajón concreto donde está guardado el material.

`<cajon>C00201</cajon>`

En algunos materiales esta etiqueta puede estar vacía, como ocurre cuando el componente está en una balda pero no dentro de un cajón concreto.

`<cajon></cajon>`

El archivo JavaScript interpreta este dato para mostrarlo en la tabla o en el CSV.

Cajon

9.9 Campo descripción

9		<code><descripcion>Placas Arduino para prácticas</descripcion></code>
---	--	---

Explicación técnica

La etiqueta descripcion guarda una explicación breve del material.

`<descripcion>Placas Arduino para prácticas</descripcion>`

Este campo ayuda a identificar el uso o tipo de componente almacenado.

En los informes generados aparece como última columna para aportar información adicional.

Descripcion

9.10 Relación con informes.js

```
2 | fetch("inventario.xml")
```

Explicación técnica

El archivo informes.js utiliza este XML como fuente de datos.

Primero carga el archivo mediante `fetch("inventario.xml")`.

```
fetch("inventario.xml")
```

Después obtiene todos los materiales con `getElementsByTagName("material")`.

```
xml.getElementsByTagName("material")
```

Finalmente extrae cada campo, como nombre, cantidad, armario, balda, cajon y descripcion, para crear una tabla HTML o un archivo CSV.

```
obtenerTexto(m, "nombre")
```

9.11 Conclusión del archivo inventario.xml

Explicación técnica

El archivo inventario.xml permite almacenar los datos del inventario de forma ordenada, sencilla y legible.

Su estructura facilita que JavaScript pueda leer automáticamente todos los materiales sin escribir los datos directamente dentro del HTML.

Gracias a este archivo, el sistema puede generar informes HTML y CSV de manera dinámica.